



RV College of
Engineering®

Go, change the world®

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Machine Learning Operations Tutorial

Forecasting No Show in Healthcare

Team Number: 6

USN

1RV23AI001

1RV23AI014

1RV23AI029

1RV23AI032

Name

Aaditey Chalva

Amudhan S

Dhaksha Muthukumaran

Dhruv Patankar



Agenda

- Introduction
- Motivation (Why this title)
- Problem Definition
- Objectives
- Data set Description
- Risk Analysis
- Tools and Techniques used
- Phase I : Machine Learning Model and its analysis
- Phase II : Building MLOPs



Introduction

- No-shows cost healthcare systems billions annually, **wasting resources** and delaying care for vulnerable patients.
- Predicting patient no-shows in healthcare optimizes resource allocation and **enhances patient care** delivery.
- This project aims to produce fair, efficient solutions that prioritize patient equity and **clinic productivity**.
- By integrating efficient models and robust pipelines, we achieve **scalable, reproducible predictions** in a real-world setting.
- End-to-end MLOps framework from data versioning to production monitoring drives sustainable impact.



Motivation

- Patient no-shows typically range from **40-50%** in outpatient settings, causing substantial financial losses and underutilized clinical capacity.
- Missed appointments **disrupt care** continuity, particularly for chronic conditions such as hypertension and diabetes, leading to poorer health outcomes.
- Public healthcare systems, serving vulnerable populations, are especially impacted by inefficient scheduling and **long waiting lists**.
- Hence, this topic was chosen as it addresses both operational efficiency and equity, aligning with sustainable and patient-centered healthcare goals.



Problem Definition

- **Problem Statement:** Develop an MLOps pipeline to predict patient no-shows in healthcare appointments using the Kaggle No-Show Appointments dataset, achieving good accuracy while ensuring scalability and reproducibility.
- No-shows lead to underutilized resources, increased wait times, and suboptimal patient outcomes in public health systems.
- Need for efficient model pipelines to handle real-time predictions without compromising ethical standards.
- Address gaps in traditional ML by incorporating versioning, tracking, and monitoring for production readiness.



Objectives

- **Business Objectives:**

- Reduce no-show rates and ensure timely follow-ups for patients
- Boost appointment slot utilization, minimizing wasted time and capacity.
- Cut missed appointments for chronic conditions to improve outcomes.

- **Technical Objectives:** Implement an end-to-end MLOps pipeline for reproducibility, scalability and continuous monitoring

- **Performance Objectives:** Attain >80% accuracy, >0.85 AUC-ROC, and <500ms inference latency; ensure fairness in classes like gender.

- **Cost Objectives:** Prioritize efficient models and lean resource usage, focusing on minimizing storage, compute and deployment costs with free/open-source tools.



Dataset Description

- **Source:** "Medical Appointment No Shows" dataset, publicly available on Kaggle - [Medical No Show](#)
- **Origin:** Real records from public healthcare facilities
- **Size:** 110,527 appointment records across 88 neighbourhoods.

Features	Patient ID, Appointment ID, Gender, Age, Neighbourhood, Scholarship, Hypertension, Diabetes, Alcoholism, Handicap, SMS_received, ScheduledDay, AppointmentDay, No-show
Categorical Features	Gender, Neighbourhood, Scholarship, Hypertension, Diabetes, Alcoholism, Handicap, SMS_received
Temporal Features	Age, ScheduledDay, AppointmentDay, PatientID, AppointmentID
Target	Binary "Show/ No-show" label



Risk Analysis

- i) Data Drift** (Probability 4, Impact 3): Seasonal changes alter patterns. Mitigate with Evidently AI detection and periodic retraining.
- ii) Model Downtime** (Probability 3, Impact 5): Server outages during peaks.
Mitigate using local script fallbacks
- iii) Bad Predictions** (Probability 4, Impact 4): False negatives cause disparities.
Mitigate with confidence checks and SHAP explainability.
- iv) Performance Degradation** (Probability 5, Impact 4): Fairness/accuracy drops over time. Mitigate via monthly Grafana-monitored recomputes.
- v) Skill Loss** (Probability 3, Impact 3): Talent departure halts maintenance. Mitigate with GitHub README and handover docs.



Risk Matrix

Probability

Highly probable (5)	5	10	15	20 (iv)	25
Probable (4)	4	8	12 (i)	16 (iii)	20
Possible (3)	3	6	9 (v)	12	15 (ii)
Unlikely (2)	2	4	6	8	10
Rare (1)	1	2	3	4	5
	Very low (1)	Low (2)	Medium (3)	High (4)	Very high (5)

Impact



Tools and Techniques

- **GitHub:** For version control and collaboration. Chosen for seamless integration with CI/CD and open-source accessibility.
- **DVC:** For data versioning and reproducibility. Selected to track large datasets like Parquet files without bloating Git, enabling rollbacks.
- **scikit-learn:** For baseline models (Logistic Regression, Random Forest, XGBoost) and preprocessing. Preferred for its simplicity, efficiency, and fairness metrics integration.
- **MLflow:** For experiment tracking and model management. Adopted to log hyperparameters, metrics, and artifacts, facilitating side-by-side comparisons.



Tools and Techniques

- **GitHub Actions:** For CI/CD pipelines. Chosen to automate testing, builds, and deployments, ensuring consistent workflows.
- **Docker + FastAPI + GCP Cloud run :** For containerization, API serving, and deployment. Selected for scalable pipelines with low latency.
- **Evidently AI + Prometheus + Grafana:** For production monitoring and visualization. Picked for drift detection, metrics visualization, and alerting to maintain model integrity.
- **Techniques:** Advanced feature engineering (temporal, historical rolling rates, cyclical encoding), hyperparameter tuning, fairness-aware evaluation.



Phase I – ML Life Cycle

- Exploratory Data Analysis
- List of algorithms chosen to build ML model
- Metrics used to evaluate the ML model
- Evaluation of ML model
- Identification of Best Model



Exploratory Data Analysis

Exploration used Pandas for summary statistics, Matplotlib for visualizations, and correlation analysis. Class imbalance and potential biases were noted early. The results of the EDA were as follows:

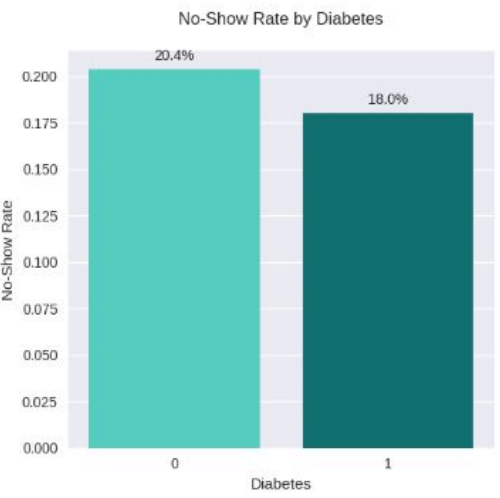
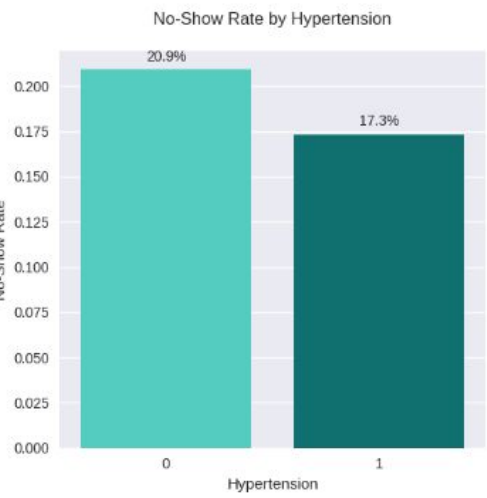
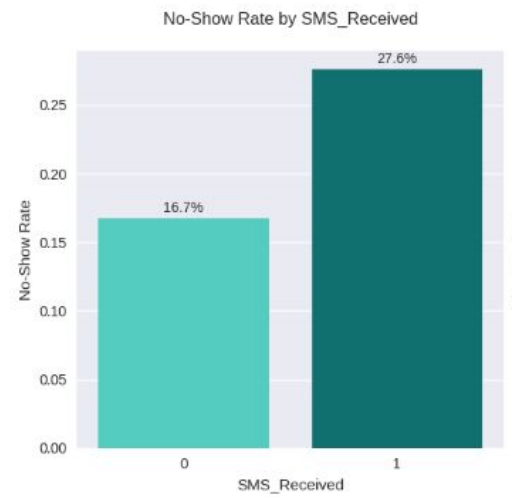
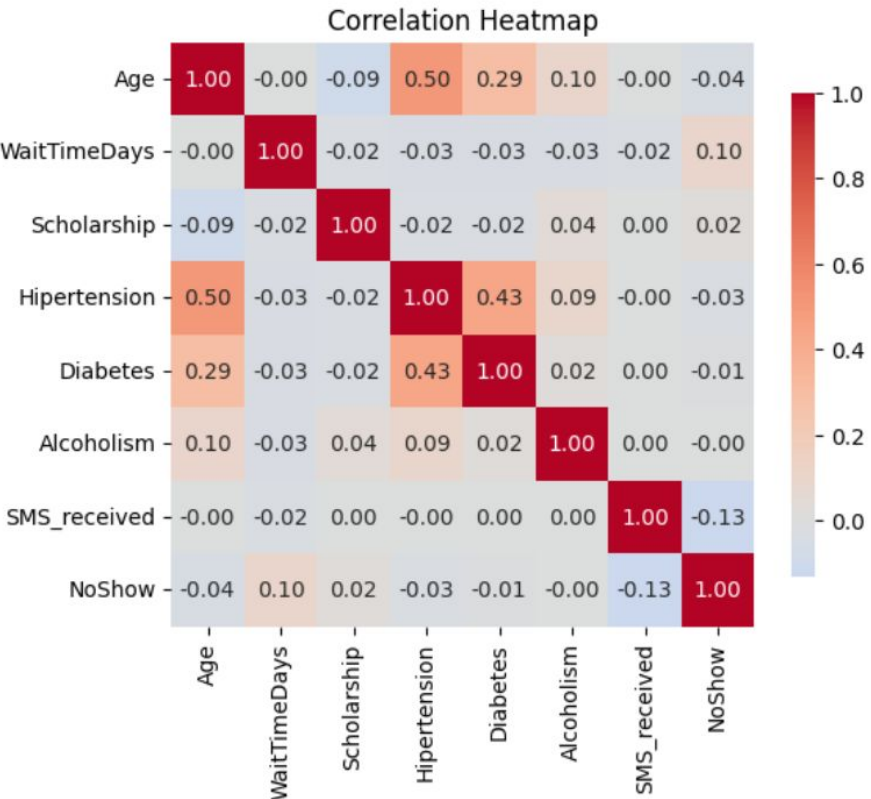
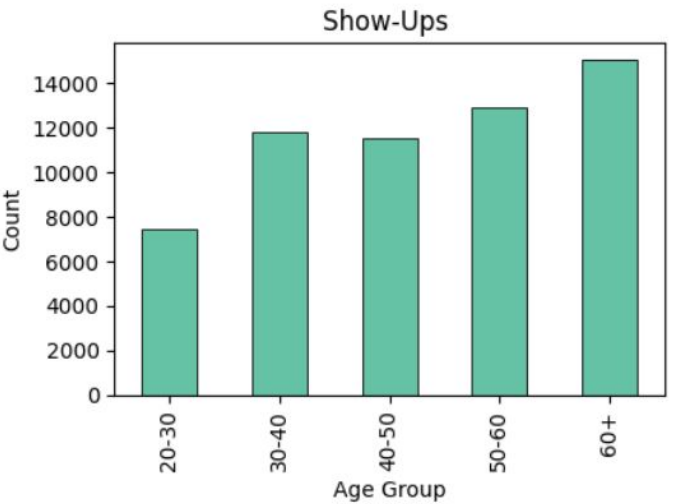
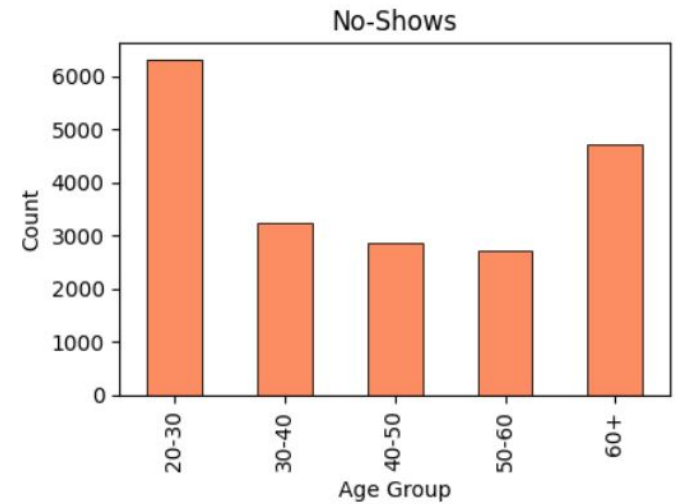
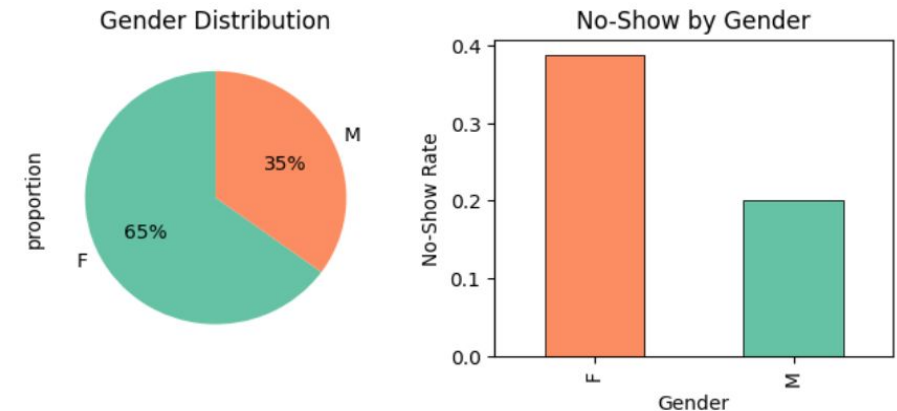
- **Gender Distribution:** 65% female, 35% male patients. Shows gender imbalance, suggesting potential gender-specific no-show patterns (e.g., females may miss more due to caregiving).
- **No-Show Rate by Scholarship:** Higher no-shows for low-SES patients, indicating Scholarship as a key predictor.
- **Age by No-Show:** No-shows peak at 20–30 years; show-ups spread broader, peaking at 50-60. Suggesting younger patients prioritize health less.
- **No-Show Rate by Day of Week:** No-shows peak midweek, drop on weekends . Shows temporal trends, indicating midweek scheduling conflicts.



Exploratory Data Analysis

- **Numerical Distributions:** Age is right-skewed. Highlight skewed distributions and outliers, guiding scaling
- **Numerical by No-Show:** Longer WaitTimeDays and younger Age correlate with no-shows; Handicap shows little variation.
- **Categorical Distributions:** Hypertension, Diabetes, SMS_received vary in prevalence. Plots reveal feature prevalence, justifying binary encoding for model input.
- **No-Show Rates by Categoricals:** SMS_received lowers no-shows; Hypertension, Diabetes slightly increase no-shows. Quantify feature impact, prioritizing SMS_received and clinical features for XGBoost feature importance.
- **Correlation Heatmap:** SMS_received and WaitTimeDays most correlated with NoShow. Identifies predictive features, supporting their emphasis in feature engineering.

Exploratory Data Analysis





Chosen Algorithms

- **Logistic Regression:** Highly interpretable, fast to train, serves as a benchmark for more complex models, handles mixed feature types well after encoding
- **Random Forest:** Robust to overfitting, handles non-linear relationships and interactions well, provides feature importance for insights, better recall on minorities.
- **XGBoost:** State-of-the-art for tabular data, excels on imbalanced classes with `scale_pos_weight`, built-in regularization prevents overfitting, fast inference for real-time use in clinics.
- Models tuned with GridSearchCV to optimize hyperparameters.



Evaluation Metrics

- **AUC-ROC:** Primary metric for ranking models, measuring discrimination ability across thresholds (>0.85 target).
- **Accuracy:** Overall correctness ($>80\%$ target), providing simple interpretability despite class imbalance.
- **Precision:** Proportion of predicted no-shows that are actual no-shows, critical for minimizing false interventions.
- **Recall:** Proportion of actual no-shows correctly identified, essential for capturing high-risk patients.
- **F1-Score:** Harmonic mean of precision and recall, balancing both in imbalanced setting.



Evaluation of Models

Model	AUC	Accuracy	Precision	Recall	F1-Score
Logistic Regression	0.6554	0.6775	0.3167	0.5143	0.3920
Random Forest	0.7088	0.7239	0.3496	0.4254	0.3838
XGBoost	0.9326	0.8400	0.8501	0.8305	0.8505

- XGBoost: AUC: 0.9326, Accuracy: 0.84, Precision: 0.85
- All evaluations conducted on hold-out test set, metrics logged in MLflow for comparison.
- SHAP values monitored across sensitive groups (e.g., age, scholarship) to ensure fairness



Identification of Best Model

- **XGBoost** identified as best model with superior AUC-ROC of 0.9326 and accuracy of 0.8400.
- Outperformed baselines: Significantly higher than Random Forest and Logistic Regression
- Achieved balanced performance: High recall for no-shows while maintaining strong precision. Effective handling of engineered features, regularization
- Selected for production: Meets performance targets with low latency potential.
- Model artifact versioned in MLflow for further deployment.



Identification of Best Model

 **MLOps Control**

v2.4.0-stable

Dashboard

Predictor

Architecture

Monitoring

Inference Engine

Manual entry interface for patient no-show prediction

Patient Parameters

Enter clinical and demographic data

Age

30

Gender

Female

Neighbourhood

Downtown

Handicap Level (0-4)

0

Scheduled Date

12/28/2025

Appointment Date

12/28/2025

Conditions & Status

Diabetes

☒

Hypertension

☒

Alcoholism

☐

Scholarship

☐

SMS Sent

☐

 Run Prediction Model

Analysis Result

Model confidence output

46%

RISK

Likely to Show Up

Based on current model parameters (v1.6), this appointment has a low probability of being missed.

Model:

XGBoost Classifier

Latency:

42ms

Threshold:

0.50

Department of AI and ML

20



Identification of Best Model

 **MLOps Control**
v2.4.0-stable

Dashboard

Predictor

Architecture

Monitoring

Inference Engine

Patient Parameters

Enter clinical and demographic data

Age

Gender

35

Female

Neighbourhood

Handicap Level (0-4)

Downtown

0

Scheduled Date

Appointment Date

12/30/2025

12/30/2025

Conditions & Status

Diabetes

Hypertension

Alcoholism

Scholarship

SMS Sent

Run Prediction Model

Analysis Result

Model confidence output

54%

RISK

High Risk of No-Show

Based on current model parameters (v1.6), this appointment has a high probability of being missed.

Model: XGBoost Classifier

Latency: 42ms

Threshold: 0.50

System Operational

Cluster: us-east-1a

Latency: 24ms

Prediction Complete

Probability calculated: 54.2%



Phase II – MLOps Life Cycle

	Component	MLOPs Tools
1	Data Management	DVC, GitHub
2	Experiment Tracking and Version Control	MLflow
3	Automation and Pipeline Orchestration	GitHub Actions
4	Model Deployment & Serving	Docker, FastAPI, GCP
5	Monitoring & Governance and collaboration	Evidently AI, Prometheus, Grafana



System Architecture

Overview of the MLOps pipeline and tech stack



Data Ingestion

CSV + DVC

Raw data validation and versioning



Model Training

XGBoost + MLflow

Experiment tracking & registry



Serving

FastAPI + Docker

Real-time inference endpoint



Version Control

Code and Data

We strictly separate code versioning (Git) from data versioning (DVC). This ensures reproducibility of experiments by linking specific data commits to model versions.

DVC

Git



Data Validation

Quality Assurance

Data quality checks ensure schema validity and statistical properties before training. Automated validation detects anomalies and drift in incoming data.

Pandas Profiling

Custom Validators



Orchestration

Workflow Management

GitHub Actions orchestrates the CI/CD pipeline: Data Validation → Model Training → Evaluation → Deployment. Automated workflows trigger on code changes and scheduled intervals.

GitHub Actions

DVC Pipelines



Monitoring

Observability

Prometheus scrapes metrics from the model endpoint. Grafana dashboards visualize request rates, latency, and prediction distribution to detect concept drift.

Prometheus

Grafana

Evidently AI

System Operational

Cluster: us-east-1a

Latency: 24ms



Data Management and Versioning

- Versioning and preprocessing dataset to ensure traceability and reproducibility using **DVC** for data versioning, GitHub for code and metadata integration.
- Raw data registered as version 1 using DVC, tracking changes in Parquet files
- Transformations include **handling outliers, deriving temporal features** (e.g., lead_time_days, hour_block) saved in sequential directories (e.g., data/02_cleaned).
- DVC generates .dvc pointer, committed to GitHub for audit trails and rollbacks via 'dvc checkout'.
- **Benefits:** Prevents overwrites, enables experimentation, and guarantees exact data replication for any commit.



Data Management and Versioning

GitHub repository link: [No Show-dvc-repo](#)

```
[main 333af16] v2: PreProcess + Initial Feature engineering
Date: Wed Nov 19 01:12:05 2025 +0000
5 files changed, 16 insertions(+)
create mode 100644 .dvc/.gitignore
create mode 100644 .dvc/config
create mode 100644 .dvcignore
create mode 100644 data/01_raw/appointments_raw.csv.dvc
create mode 100644 data/02_cleaned/appointments_v1.parquet.
dvc
4 files changed, 20 insertions(+)
create mode 100644 data/02_cleaned/appointments_v1.parquet.
dvc
create mode 100644 data/02_cleaned/metadata1.json.dvc
create mode 100644 data/03_features/appointments_v2.parque
t.dvc
create mode 100644 data/03_features/metadata2.json.dvc
```

Fairness snapshot saved!

```
{
  "date": "2025-11-19",
  "data_version": "data/03_features/appointments_v2.parque
t",
  "target_column_used": "NoShow",
  "note": "Using true labels as predictions \u2192 DPD = 0.0
means perfect baseline fairness",
  "demographic_parity_difference": {
    "Scholarship": 0.03929,
    "Gender": 0.00347,
    "AgeBin": 0.0697,
    "Hipertension": 0.03602,
    "Diabetes": 0.0236,
    "Handcap": 0.1541
  },
  "interpretation": "Higher DPD = more bias in raw data (e.
g., Scholarship ~0.15 is typical and ethically concerning)"
}
```

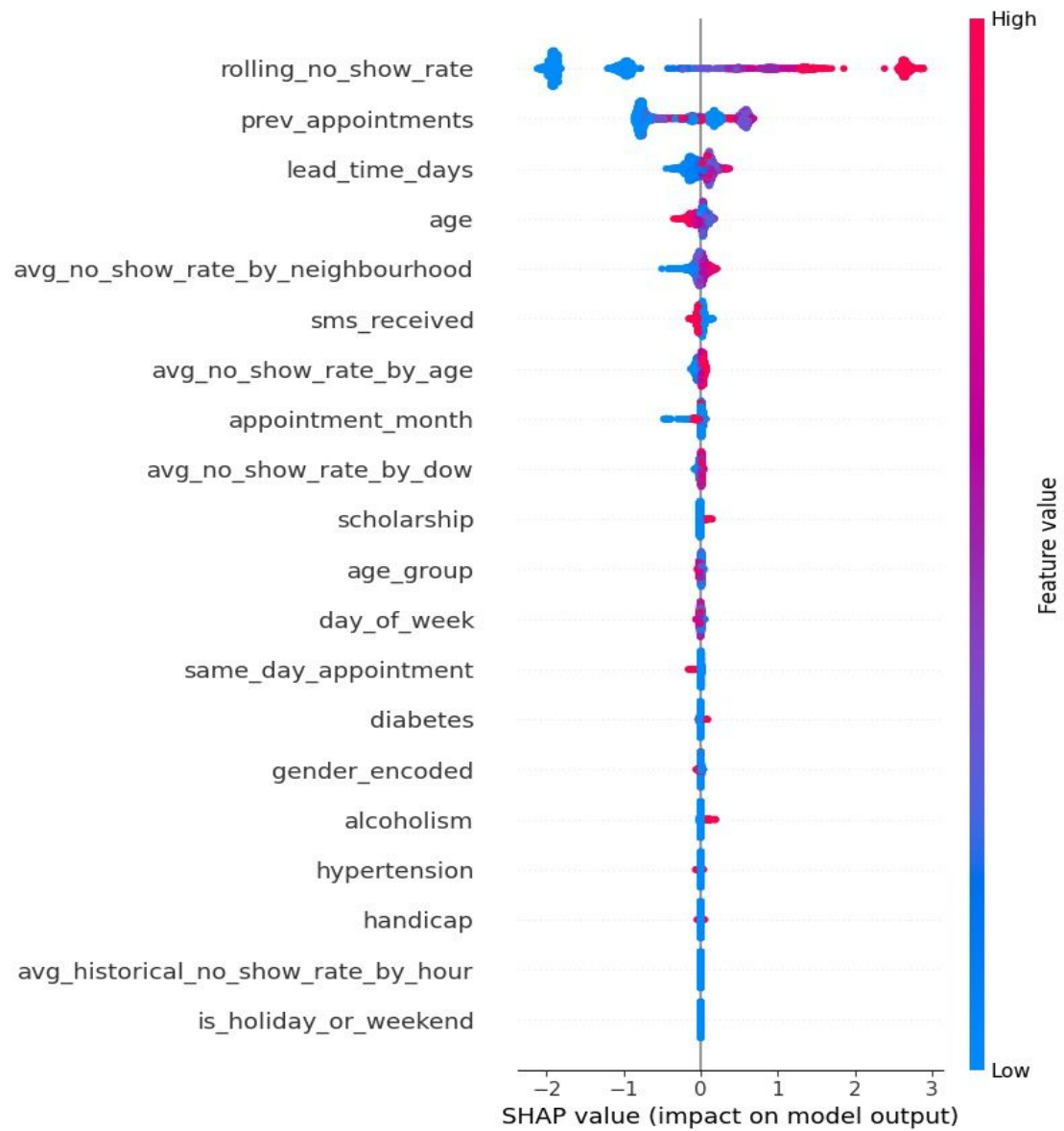


Experiment Tracking and Versioning

- Systematic logging and comparison of models to identify optimal configurations and ensure reproducibility. **MLflow** for tracking, GitHub for code versioning.
- MLflow captures hyperparameters (e.g., `n_estimators = 300` for XGBoost), metrics (AUC, accuracy), and artifacts like SHAP plots.
- UI enables **side-by-side analysis** of Logistic Regression, Random Forest, and XGBoost experiments.
- Process: Train models, log via MLflow, compare in UI, version code/artifacts.
- **Benefits:** Objective model selection, instant insights, full auditability, transparency, reproducibility, faster development.



Experiment Tracking and Versioning



Experiments >

Displaying Runs from 4 Experiments

metrics.rmse < 1 and params.model = "tree"

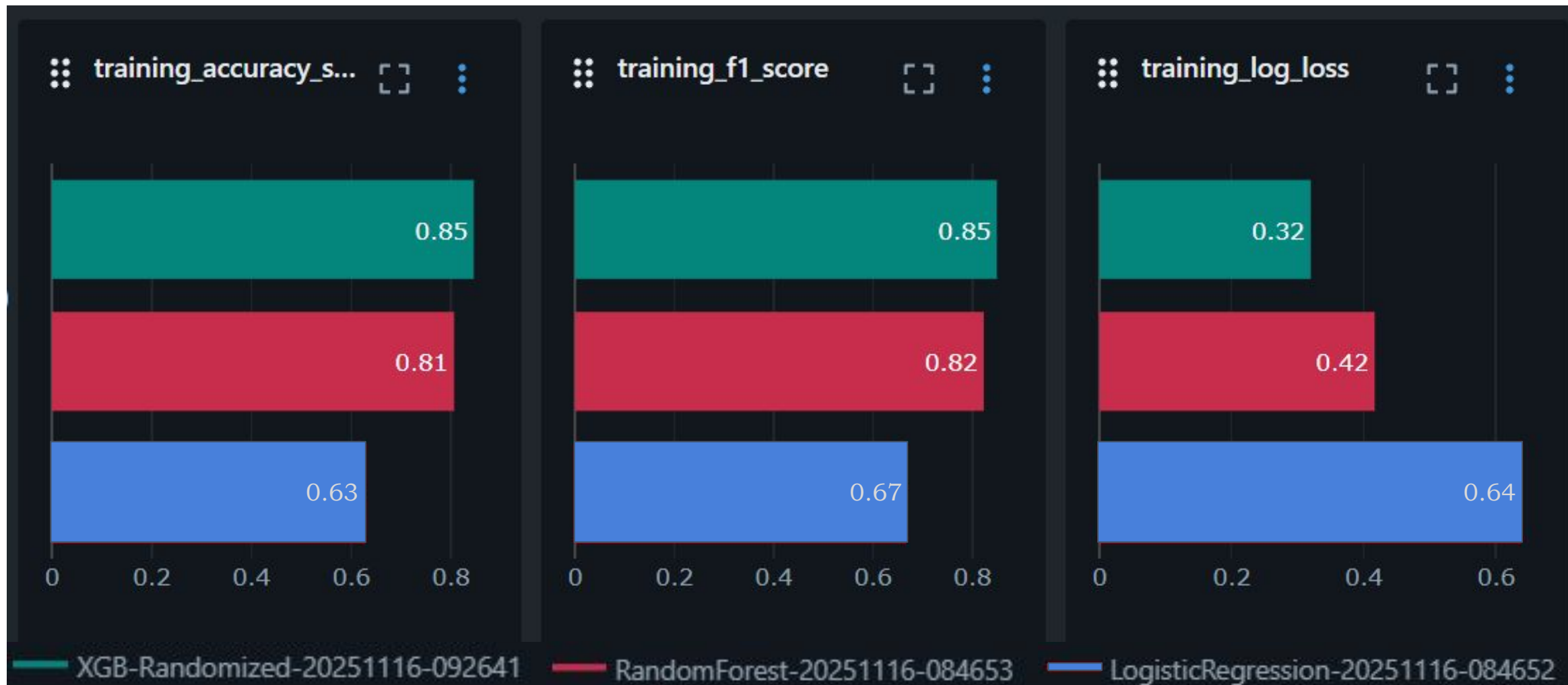
Sort: Created Columns Expand rows Group by

	Run Name	Created	Duration	Tags
☐	XGB-Randomized-20...	1 month ago	1.5min	Randomized...
☐	xgb_shap_20251116-...	1 month ago	2.3s	GridSearchCV
☐	xgb_shap_20251116-...	1 month ago	2.6s	GridSearchCV
☐	xgb_shap_20251116-...	1 month ago	2.2s	GridSearchCV
☐	xgb_shap_20251116-...	1 month ago	1.6s	GridSearchCV
☐	RandomForest-2025...	1 month ago	9.5s	RandomFore...
☐	LogisticRegression-2...	1 month ago	0.8s	LogisticRegr...
☐	Random Forest	1 month ago	9.5s	-

14 matching runs



Experiment Tracking and Versioning





Automation & Pipeline Orchestration

- Automates **end-to-end workflows** from data processing to model training and deployment for efficiency and consistency.
- **GitHub Actions** handles CI/CD, triggering tests, builds, and deployments on code pushes for rapid iteration.
- Process: Actions for sequential execution (e.g., source → test → deploy)
- Ensures reproducibility, supports scalability and transparency for production-grade pipelines.
- **Benefits:** Reduces manual errors, enables scheduled runs (e.g., monthly retraining), and facilitates integration with other tools.

Model Overview

Real-time metrics for No-Show Prediction Model (XGBoost)

Model Accuracy



89.2%

+2.4% vs last version

Total Predictions



23

All time

Pipeline Status



Healthy

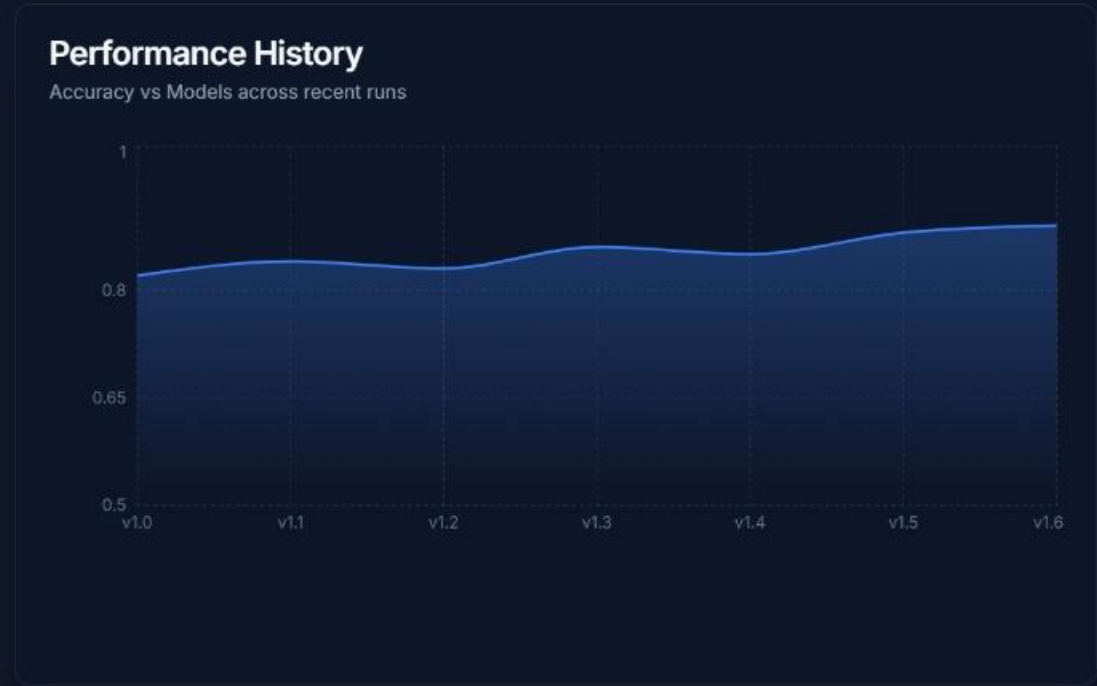
Last run: Dec 28, 20:05

Drift Detected



None

Kolmogorov-Smirnov Test



Pipeline Activity

Recent execution logs

run-1766932514815

Dec 28, 20:05

running

run-1766930828460

Dec 28, 19:37

completed

run-1766930823443

Dec 28, 19:37

running

run-1766928844702

Dec 28, 19:04

completed

run-1766928839684

Dec 28, 19:03

running

Recent Inference Requests

Live incoming predictions from the serving endpoint

Time	Age	Gender	Conditions	Probability	Result
12:02:22	35	F	Hyp	54.2%	No Show
12:02:10	35	F	-	53.2%	No Show



Model Deployment & Serving

- **Docker** containerizes the application, packaging dependencies and the model (XGBoost) for consistent environments.
- **FastAPI** serves predictions via APIs, handling requests with low latency (<500ms) for no-show probability outputs.
- **GCP Cloud Run** runs containerized models and automatically scales for more requests
- Process: Build Docker images, expose via FastAPI endpoints, deploy via GCP
- **Benefits:** Ensures fault tolerance and scaling to handle varying loads in healthcare systems.



Model Deployment & Serving

Summary

All jobs

Build and Deploy to Cloud Run

Run details

Usage

Workflow file

Build and Deploy to Cloud Run

succeeded 2 days ago in 4m 56s

Search logs

> ✓ Checkout code

1s

> ✓ Authenticate to Google Cloud

0s

> ✓ Set up Cloud SDK

23s

> ✓ Configure Docker for Artifact Registry

1s

> ✓ Build Docker image

1m 46s

> ✓ Push Docker image to Artifact Registry

1m 14s

> ✓ Deploy to Cloud Run

1m 24s

> ✓ Get Cloud Run URL

3s

> ✓ Test deployment

0s

> ✓ Output deployment info

0s

> ✓ Post Authenticate to Google Cloud

0s

> ✓ Post Checkout code

0s

> ✓ Complete job

1s

Department of AI and ML

32



Model Deployment & Serving

docker.desktop PERSONAL

Ctrl+K

?

4

Sign in

To access the latest features, sign in

Containers

Images

Volumes

Builds

Docker Hub

Docker Scout

Extensions

Images [Give feedback](#)

View and manage your local and Docker Hub images. [Learn more](#)

Local Docker Hub repositories

43.41 MB / 17.93 GB in use 2 images

Last refresh: 52 seconds ago

☐	Name	Tag	Image ID	Created	Size	Actions
☐	redis	7-alpine	ee64a64eaab6	2 months ago	60.66 MB	
☐	noshow-predictor	test	e471aaa31a28	3 days ago	2.48 GB	

Showing 2 items

Engine running

RAM 1.53 GB CPU 0.03% Disk: 24.24 GB used (limit 1006.85 GB)

Terminal

New version available



Monitoring & Visualization

- Continuous oversight of model performance, drift, and fairness post-deployment.
- **Evidently AI** detects data/model drift to ensure that model performs as expected
- **Prometheus** collects metrics for monitoring and alerting; **Grafana** visualizes dashboards for accuracy, latency, and alerts.
- Governance covered by bias monitoring (Evidently AI), SHAP explainability, anonymized logs, clinician oversight and structured checks for compliance
- Collaboration via GitHub for tracking, and shared repositories; promotes transparency with versioned artifacts.



Monitoring & Visualization

MLOps Control
v2.4.0-stable

Dashboard

Predictor

Architecture

Monitoring

System Monitoring

Real-time status of all MLOps components

System Health

100%

6 of 6 active

Components

6

Total configured

Last Updated

9:58:37 PM

All systems operational

DVC (Data Version Control) Active

Initialized: ✓

Output:

Tracked Files: 2

Pipeline Stages:

- data_validation

Evidently AI Active

Installed: ✓

Reports Generated: 1

Latest Report: drift_report.html

Monitoring Active: ✓

Drift Calculation: Statistical distance-based

Prometheus Active

Installed: ✓

Config Exists: ✓

Registered Metrics: 7

Alert Rules Configured: ✓

Metrics Endpoint: /metrics

Grafana Active

Dashboard Configured: ✓

Dashboard Title: ML Model Monitoring

Panel Count: 4

Docker Active

Dockerfile Exists: ✓

Docker Daemon Running: ✓

Dockerfile Path: docker/Dockerfile

GitHub Actions Active

Configured: ✓

Workflow Count: 4

Workflows:

- ci.yml
- deploy-gcp.yml
- model-promotion.yml
- model-training.yml

● System Operational

Cluster: us-east-1a

Latency: 24ms



Monitoring & Governance

Prometheus Alerts Graph Status ▾ Help

Targets

All scrape pools ▾ All Unhealthy Collapse All

Q

Filter by endpoint or labels

☒ Unknown

☒ Unhealthy

☒ Healthy

ml_model_api (1/1 up)

show less

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://localhost:5000/api/metrics	UP	instance="localhost:5000" job="ml_model_api" ▾	22.787s ago	4.463ms	

prometheus (1/1 up)

show less

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://localhost:9090/metrics	UP	Instance="localhost:9090" job="prometheus" ▾	1.125s ago	8.535ms	



**RV College of
Engineering®**

Mysore Road, RV Vidyaniketan Post,
Bengaluru - 560059, Karnataka, India

Go, change the world

THANK YOU